

Aula 05

API REST: o CRUD completo

Criar · Ler · Atualizar · Apagar

O que é REST?

Não é tecnologia. Não é biblioteca.

É um **acordo sobre como nomear as coisas**,
para que qualquer dev entenda sua API sem manual.

A URL diz **qual coisa**

O método diz **o que fazer**

Compare

 Sem REST	 Com REST
GET /listarTarefas	GET /tarefas
GET /pegarTarefa?id=5	GET /tarefas/5
POST /criarNovaTarefa	POST /tarefas
POST /atualizarTarefa	PUT /tarefas/5
GET /deletarTarefa?id=5	DELETE /tarefas/5

Na esquerda você **decora** cada rota.

Na direita o padrão **se repete** para qualquer recurso.

4 regras de nomenclatura

1. **Substantivo, plural, minúsculo** → `/tarefas`
2. **Sem verbo na URL** → o verbo é o método
3. **Hierarquia com barra** → `/usuarios/3/tarefas`
4. **Filtro é query** → `/tarefas?concluida=true`

O CRUD

Op	Método	Rota	Status
Create	POST	/tarefas	201
Read	GET	/tarefas	200
Read	GET	/tarefas/:id	200
Update	PUT	/tarefas/:id	200
Update	PATCH	/tarefas/:id	200
Delete	DELETE	/tarefas/:id	204

Por que isso importa tanto

Banco · rede social · e-commerce · seu app de notas

No fundo, **todos são um CRUD**
com regras de negócio em cima.

Dominar esse esqueleto = 80% do back-end

Para receber dados: 1 linha obrigatória

```
const app = express();  
app.use(express.json()); // ⚠
```

Sem ela: `req.body` é `undefined`

 **O erro nº 1 desta aula**

O que é um middleware?

Uma função que roda **antes** das rotas.



É o porteiro do prédio

Toda visita passa por ele antes do apartamento.

Ele confere, anota, ou barra.

`express.json()` traduz o pacote JSON
que chegou em um objeto JavaScript.

POST — criando

```
app.post("/tarefas", (req, res) => {  
  const { titulo } = req.body;  
  
  if (!titulo?.trim()) {  
    return res.status(400)  
      .json({ erro: "titulo é obrigatório" });  
  }  
  
  const nova = { id: proximoId++, titulo,  
                 concluida: false };  
  tarefas.push(nova);  
  
  res.status(201).json(nova);  
});
```

3 decisões importantes no POST

1. Valide sempre

Nunca confie no que chega de fora

2. O **servidor** gera o id

Nunca o cliente

3. Devolva o objeto criado

O cliente precisa saber o id

PUT × PATCH

	PUT	PATCH
Significa	substitua tudo	altere um pedaço
Corpo	todos os campos	só o que muda
Campo faltando	é apagado	fica como estava

PATCH é o mais usado no dia a dia.

PATCH na prática

```
const atualizada = {  
  ...tarefas[indice],           // tudo que já era  
  ...(titulo !== undefined && { titulo }),  
  ...(concluida !== undefined && { concluida }),  
};
```

O spread da aula 02, trabalhando por você.

Entendendo o spread condicional

```
...(titulo !== undefined && { titulo })
```

Passo a passo:

Se <code>titulo</code> veio	<code>&&</code> devolve	espalhar isso
sim	<code>{ titulo: "x" }</code>	adiciona o campo
não	<code>false</code>	não faz nada

Espalhar `false` num objeto é inofensivo.

É um truque — se achar confuso, escreva com `if`.

Métodos de array que **mutam**

Aqui, no back-end, mutar é aceitável:

```
tarefas.push(nova);           // adiciona no fim  
tarefas.splice(indice, 1);    // remove 1 item na posição  
tarefas[indice] = atualizada; // substitui
```

| `.push(item)` | adiciona no fim, devolve o novo tamanho |

| `.splice(i, n)` | remove `n` itens a partir de `i` |

⚠ No React (aula 08) **nada disso pode ser usado.**

! slice × splice

	O que faz	Altera?
<code>.slice(i, f)</code>	copia um pedaço	✗ não
<code>.splice(i, n)</code>	corta do original	● sim

Uma letra de diferença, comportamentos opostos.

```
[1,2,3].slice(0, 2)    // [1, 2] – original intacto  
[1,2,3].splice(0, 2)   // original vira [3]
```



`new Date().toISOString()`

```
new Date()           // agora  
new Date().toISOString() // "2026-04-01T13:45:30.123Z"
```

Sempre guarde datas nesse formato. Por quê?

- ordena corretamente como texto
- não tem ambiguidade dia/mês
- é entendido no mundo todo

`"01/04/2026"` é 1º de abril aqui
e 4 de janeiro nos EUA. 🤪

`Object.keys()` para detectar vazio

```
Object.keys({ a: 1, b: 2 }) // ["a", "b"]  
Object.keys({})           // []
```

```
if (Object.keys(req.body).length === 0) {  
  return res.status(400)  
    .json({ erro: "informe algum campo" });  
}
```

Um PATCH com corpo vazio não faz sentido.

⚠ Nunca use `if (concluida)`

```
if (concluida) { ... } // ✖  
if (concluida !== undefined) { ... } // ✔
```

Se o cliente mandar `concluida: false`,
a primeira condição é **falsa** e o campo é ignorado.

Bug clássico. Difícil de achar.

DELETE

```
app.delete("/tarefas/:id", (req, res) => {  
  const indice = tarefas.findIndex(t => t.id === id);  
  
  if (indice === -1) {  
    return res.status(404)  
      .json({ erro: "não encontrada" });  
  }  
  
  tarefas.splice(indice, 1);  
  res.status(204).end();  
});
```

204 = "deu certo, e não tenho nada a devolver"

Validação que escala

```
function validarTarefa(dados) {  
  const erros = [];  
  if (!dados.titulo?.trim())  
    erros.push("titulo é obrigatório");  
  if (dados.titulo?.length > 100)  
    erros.push("titulo: máximo 100 caracteres");  
  return erros;  
}  
  
const erros = validarTarefa(req.body);  
if (erros.length) return res.status(400).json({ erros });
```

Devolva todos os erros de uma vez.

Consistência de respostas

```
// lista
{ "total": 3, "dados": [...] }

// um item
{ "id": 1, "titulo": "..." }

// erro
{ "erro": "Tarefa 99 não encontrada" }

// vários erros
{ "erros": ["titulo é obrigatório", ...] }
```

Não existe formato certo.

Existe formato **consistente**.

Teste o ciclo completo

POST	/tarefas	→ cria (201)
GET	/tarefas	→ apareceu?
PATCH	/tarefas/5	→ marca concluída
DELETE	/tarefas/5	→ apaga (204)
GET	/tarefas/5	→ sumiu? (404)

E teste os caminhos infelizes

- criar sem título
- atualizar id que não existe
- mandar prioridade inválida
- mandar id como texto

Dev júnior testa o caminho feliz.

Dev sênior testa o resto.

Lembre-se

Os dados vivem **em memória**.

Reiniciou o servidor → tudo volta ao início.

Na aula 07 isso vira um **banco de dados**.

Recapitulando

1. URL = **qual coisa** · método = **o que fazer**
2. `express.json()` é obrigatório para ler `req.body`
3. Valide tudo · servidor gera o id · status correto sempre

Mão na massa 🚀

`aula-05/exercicios/README.md`

Próxima: seu `server.js` tem 150 linhas.

E com 20 rotas? → **camadas**